# BLUE WATER / MSRX

# MSRX → SUPER TRANSFER

## Complete End-to-End Architecture & Worker Processing Flow

Developer KT / Onboarding

Source: `23_MSRX_SUPER_TRANSFER_VISUAL_DIAGRAMS.md`
All Mermaid diagrams rendered visually — no raw Mermaid source as diagrams.
UNKNOWN / KT CONFIRMATION REQUIRED boxes preserved from audit.

| | |
|---|---|
| **CONFIRMED FROM CODE / CONFIG** | Solid boxes and normal arrows |
| **UNKNOWN — KT CONFIRMATION REQUIRED** | ??? boxes — do not invent architecture |

## Document Contents

**Diagram 1.** COMPLETE END-TO-END MASTER FLOW (Tall portrait)
**Diagram 2.** FRONTEND → DJANGO (Confirm Commit Sequence) (Landscape)
**Diagram 3.** DJANGO `confirm_commit` INTERNAL FLOW (Tall portrait)
**Diagram 4.** `seller-loan_id` CREATION (Landscape)
**Diagram 5.** PRODUCER / CONSUMER GAP (Tall portrait)
**Diagram 6.** WORKER STARTUP (Tall portrait)
**Diagram 7.** SQS POLLING / DECISION TREE (Tall portrait)
**Diagram 8.** `processLoan` INTERNAL PIPELINE (Tall portrait)
**Diagram 9.** CLASSIFICATION — OFFLINE TRAINING vs RUNTIME (Tall portrait)
**Diagram 10.** SEGREGATION (ILLUSTRATIVE EXAMPLE) (Landscape)
**Diagram 11.** EXTRACTION via `return_ExtractFunctions` (Landscape)
**Diagram 12.** DATA STORE INTERACTION (Worker-Centered) (Landscape)
**Diagram 13.** ONE-LOAN EXAMPLE — `117` / `42` / `2208066387` (Tall portrait)
**Diagram 14.** FAILURE FLOW (Confirmed Behavior Only) (Landscape)
**Diagram 15.** HIGH-LEVEL SYSTEM MAP (KT / Onboarding) (Landscape)
HOW TO READ THESE DIAGRAMS

## Legend — CONFIRMED vs UNKNOWN

**Solid boxes + normal arrows** — **`[CONFIRMED FROM CODE]`** or **`[CONFIRMED FROM CONFIG]`**

**Dashed / `???` / bold UNKNOWN box** — **`[UNKNOWN — KT CONFIRMATION REQUIRED]`** — do not treat as implemented

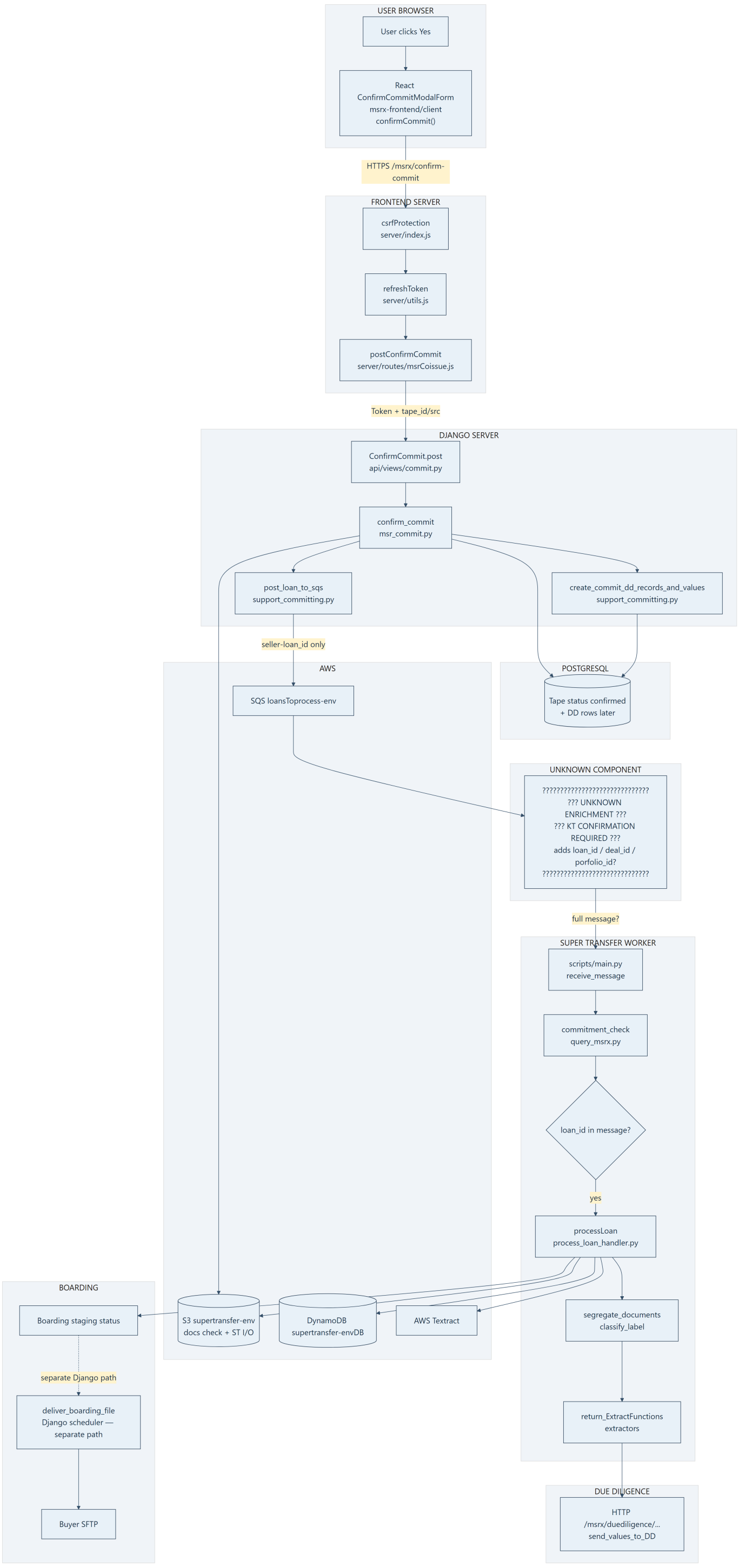
**`??? KT CONFIRMATION REQUIRED ???`** — Gap that must be asked in KT — **not** labeled Lambda unless proven

**Rule:** If a box says UNKNOWN, the audit did **not** find that component in `msrx-frontend`, `msrx_v2.0`, or `super_transfer_client`.

**Rule:** If a box says UNKNOWN, the audit did **not** find that component in msrx-frontend, msrx_v2.0, or super_transfer_client.

# Diagram 1. COMPLETE END-TO-END MASTER FLOW

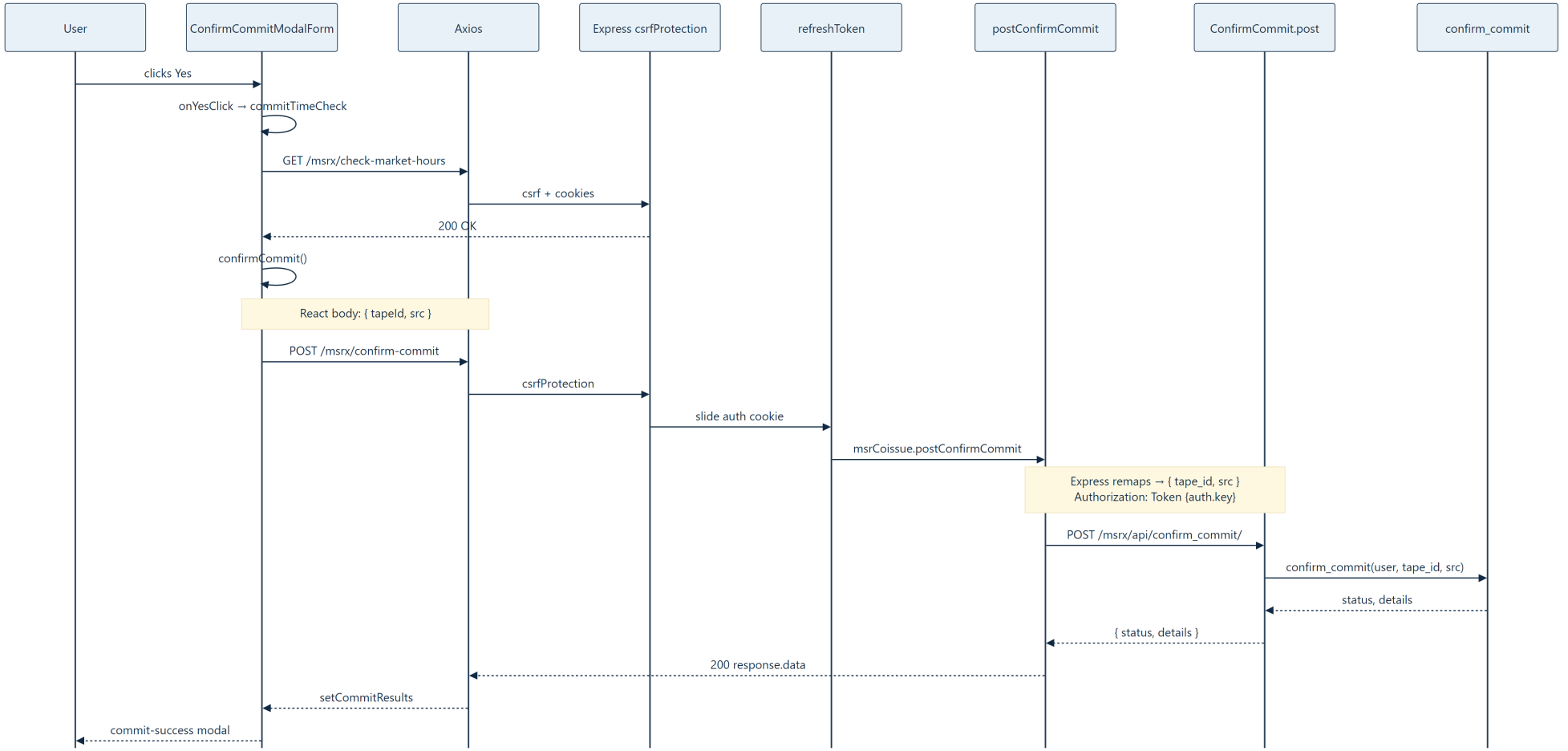
Confirm commit starts in the browser, passes Express → Django, may enqueue SQS, then (after an unknown enrichment step) the Super Transfer worker classifies and extracts documents and posts Due Diligence / boarding updates. Boarding Excel → buyer SFTP is a **separate** Django scheduler path, not inside the worker's sync HTTP response. The UNKNOWN box between SQS and the worker is intentional — producer and consumer schemas do not match in these repos.




**KEY TAKEAWAY:** Django enqueues only `seller-loan_id`; the worker needs `loan_id` before `processLoan`. The component that bridges that gap is **unknown** — do not invent it.

# Diagram 2. FRONTEND → DJANGO (Confirm Commit Sequence)

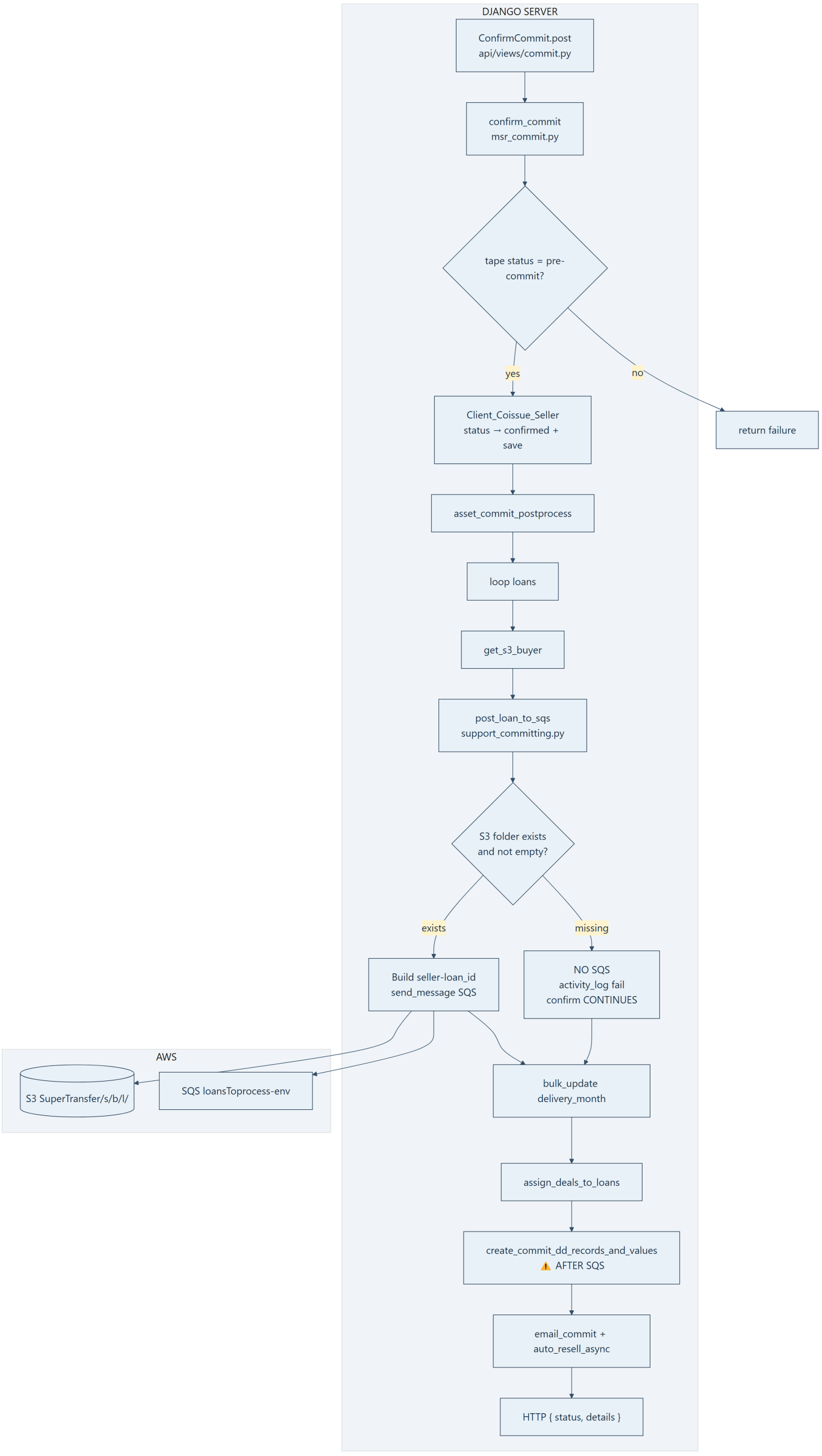
The MSR Coissue wizard uses **direct axios**, not `apiHoc`. Express is a hand-rolled BFF: CSRF, cookie session slide, body remap (`tapeId` → `tape_id`), and `Authorization: Token`. React never holds the auth token — it lives in an httpOnly signed cookie. Confirm success opens the commit-success modal; it does **not** wait for Super Transfer.




**KEY TAKEAWAY:** Express remaps and authenticates; Django owns commit business logic. UI success ≠ Super Transfer complete.

## Diagram 3. DJANGO `confirm_commit` INTERNAL FLOW

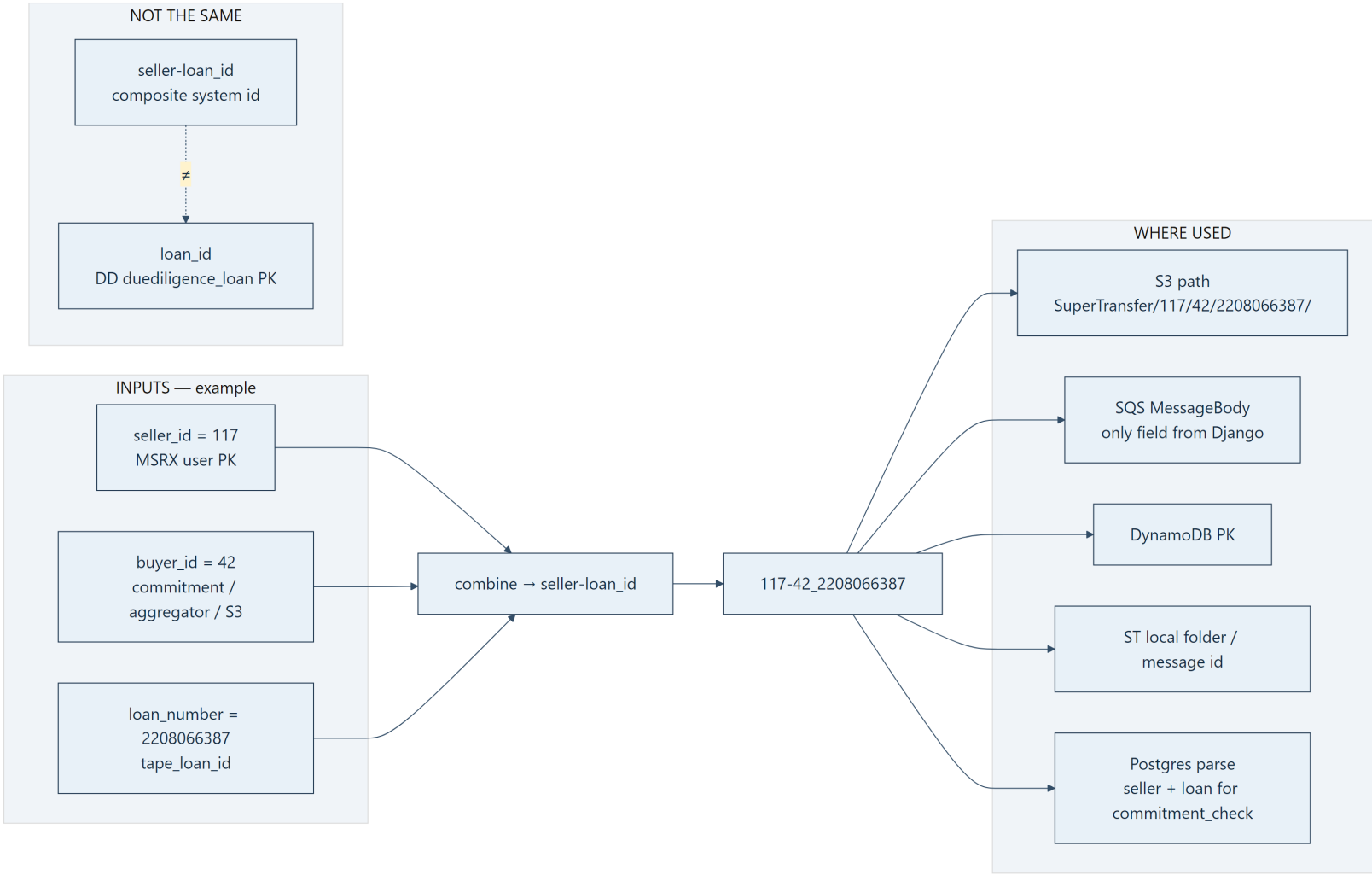
`ConfirmCommit.post` calls `confirm_commit`. After status → `confirmed` and loan loop work, `post_loan_to_sqs` **runs before** `create_commit_dd_records_and_values`. If S3 docs are missing, SQS is skipped but confirm still continues. SQS send failure does not roll back confirm — only activity log.




**KEY TAKEAWAY: SQS is sent before DD `loan_id` rows are created.** If enrichment needs that PK, this ordering is a race risk.

## Diagram 4. `seller-loan_id` CREATION

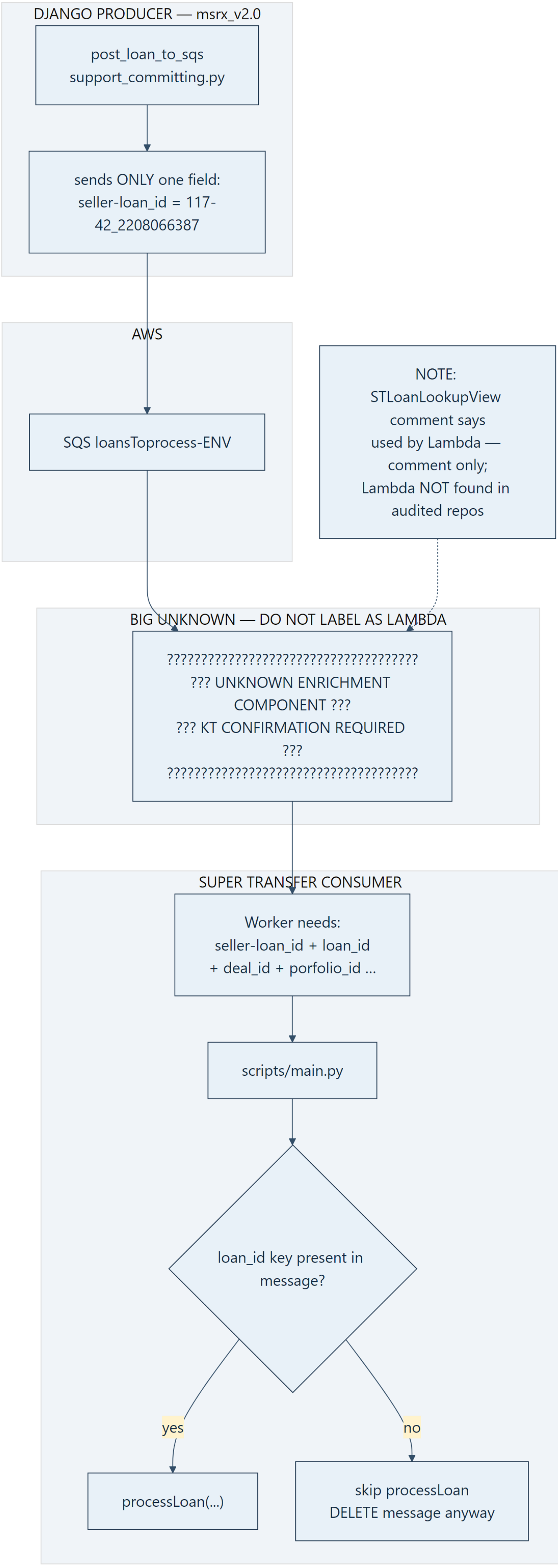
Format is `{seller_id}-{buyer_id}_{loan_number}` — verified in `post_loan_to_sqs` and worker parsers (`split("-")` then `split("_")`). Same composite string keys S3 folders, SQS messages, DynamoDB, and local worker dirs. It is **not** the Due Diligence integer `loan_id`.




**KEY TAKEAWAY:** `117-42_2208066387` is the cross-system identity. DD `loan_id` is a different integer PK required by the worker gate.

## Diagram 5. PRODUCER / CONSUMER GAP

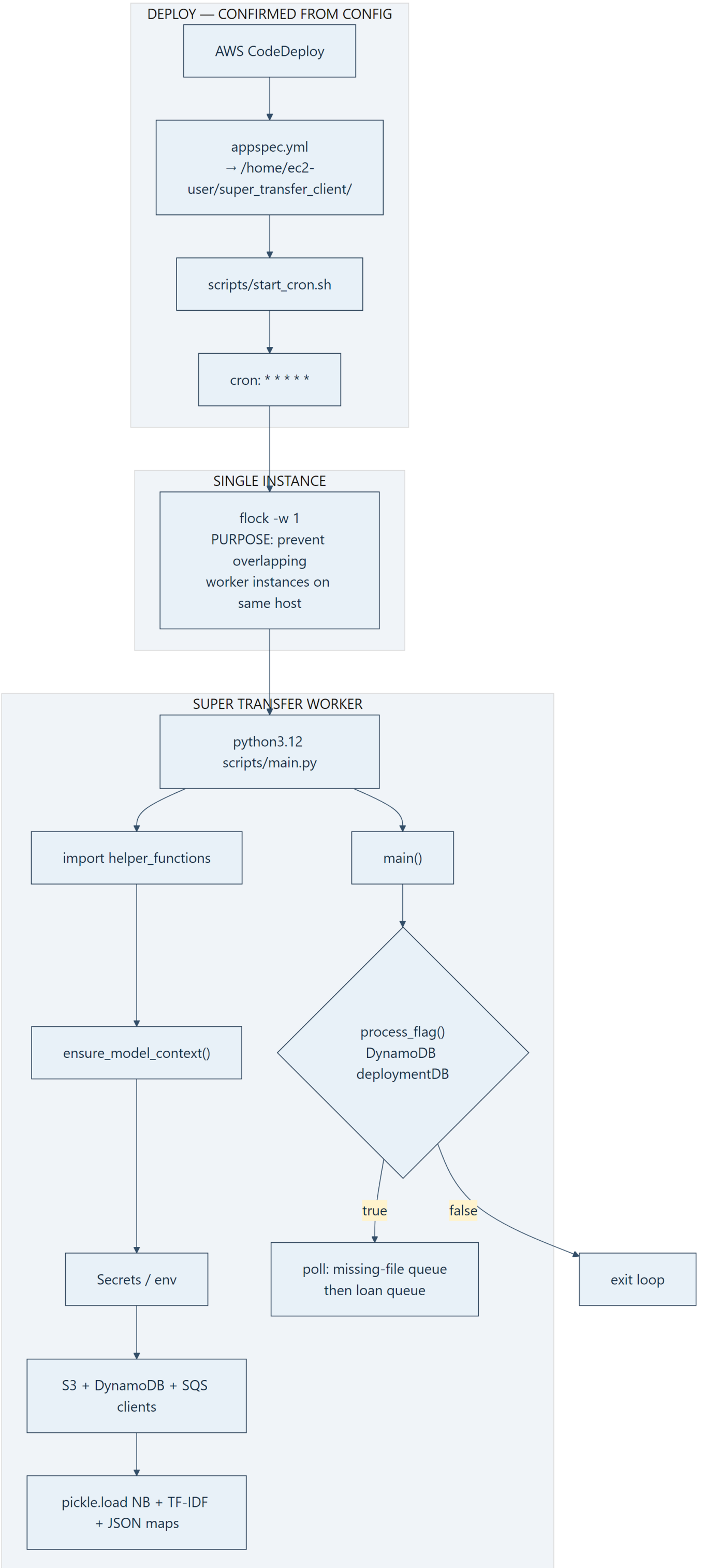
Django's producer sends **only** `seller-loan_id`. The worker's `main.py` requires `'loan_id' in res` before calling `processLoan`. No enrichment Lambda (or other bridge) exists in the three audited repos. `STLoanLookupView`'s docstring mentions Lambda — comment only; worker does not call that view.




**KEY TAKEAWAY:** Schema mismatch is proven. What fills `loan_id` in production is a **KT question** — not a silent Lambda assumption.

## Diagram 6. WORKER STARTUP

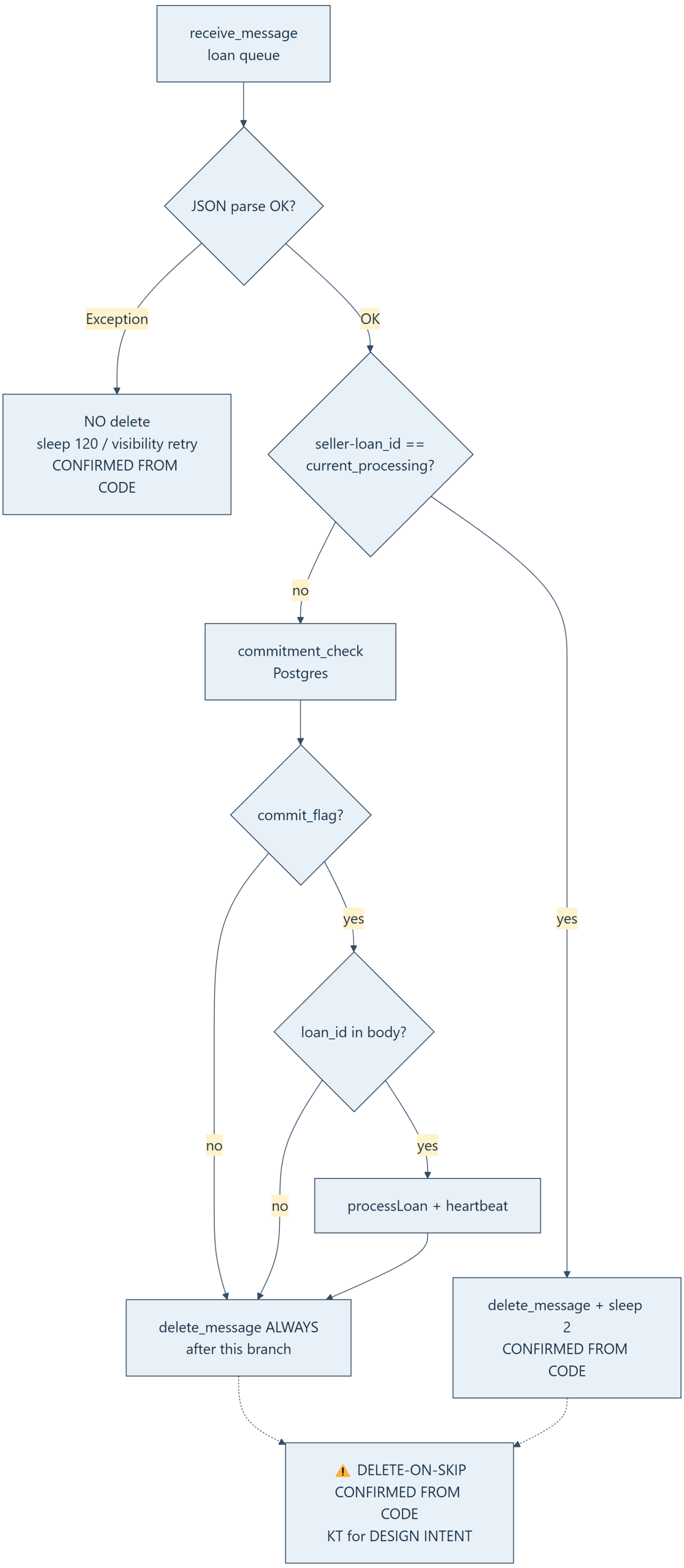
CodeDeploy installs to `/home/ec2-user/super_transfer_client/`. `start_cron.sh` installs a per-minute crontab that runs `flock` then `python3.12 scripts/main.py`. `flock -w 1` ensures a single overlapping worker process on the machine. `ensure_model_context` loads secrets, AWS clients, and ML pickles once at import; `main()` loops while `process_flag()` is true.




**KEY TAKEAWAY:** Worker is cron + flock, not systemd/supervisor (absent in repo). Models load at startup — not trained per loan.

## Diagram 7. SQS POLLING / DECISION TREE

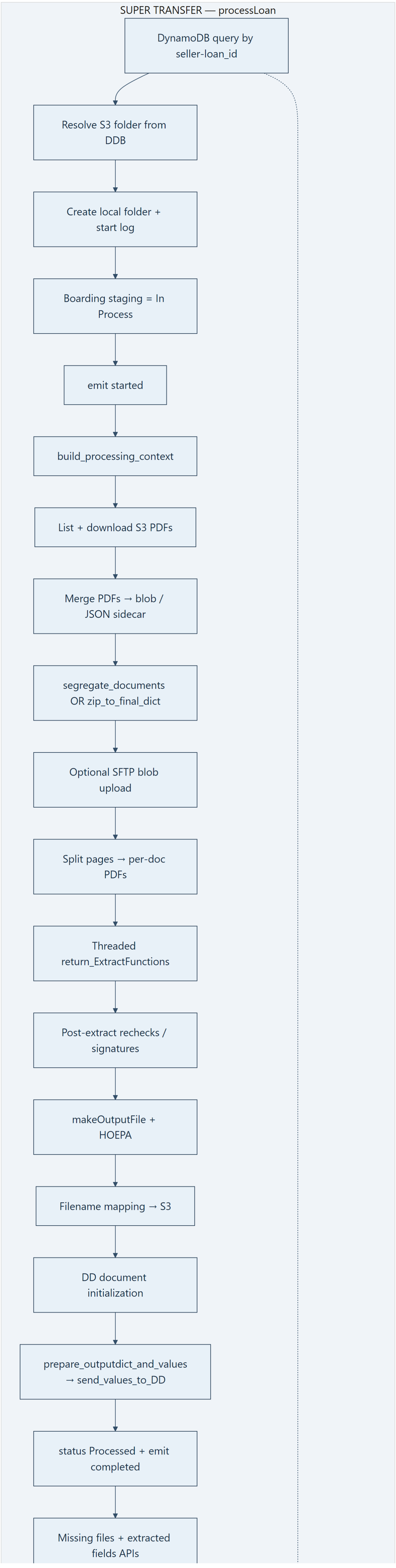
`main.py` long-polls (`WaitTimeSeconds=20`, `MaxNumberOfMessages=1`). Malformed JSON does **not** delete (visibility retry). Duplicate `current_processing`, failed `commitment_check`, missing `loan_id`, and completed `processLoan` all **delete** the message. Delete-on-skip is **confirmed from code**; whether that is intentional product design needs **KT**.




**KEY TAKEAWAY:** Missing `loan_id` or failed commit check → **no processLoan, message still deleted** — high data-loss risk until KT confirms intent / recovery.

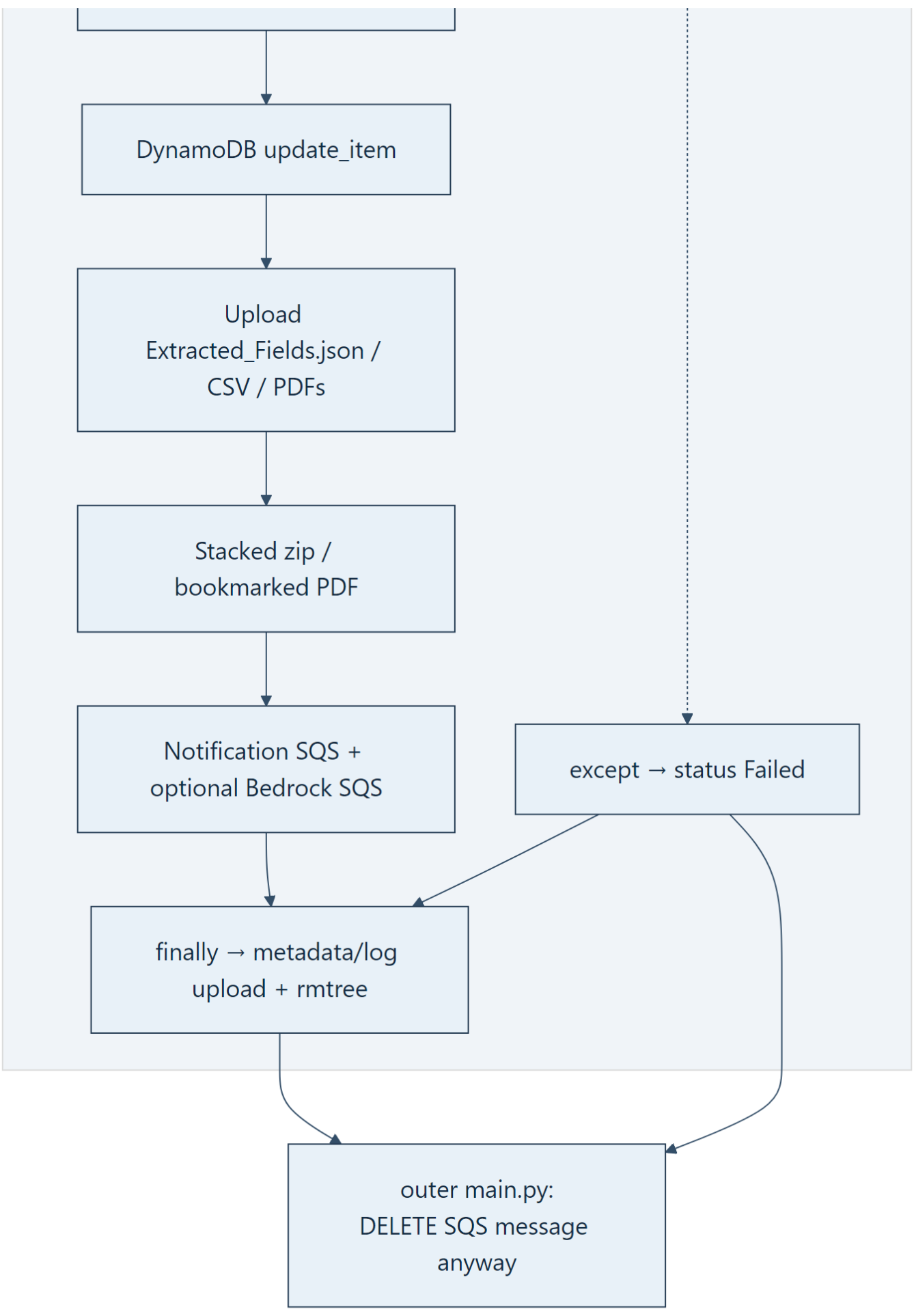
## Diagram 8. `processLoan` INTERNAL PIPELINE — Part 1/2

Real order from `process_loan_handler.processLoan`: DynamoDB → S3 download → blob → segregate → extract → DD → status → DDB update → S3 outputs → notifications. Exceptions set status `"Failed"` and are **not** re-raised; outer `main.py` still deletes SQS. `finally` uploads metadata/logs and removes the local folder.



## Diagram 8. `processLoan` INTERNAL PIPELINE — Part 2/2

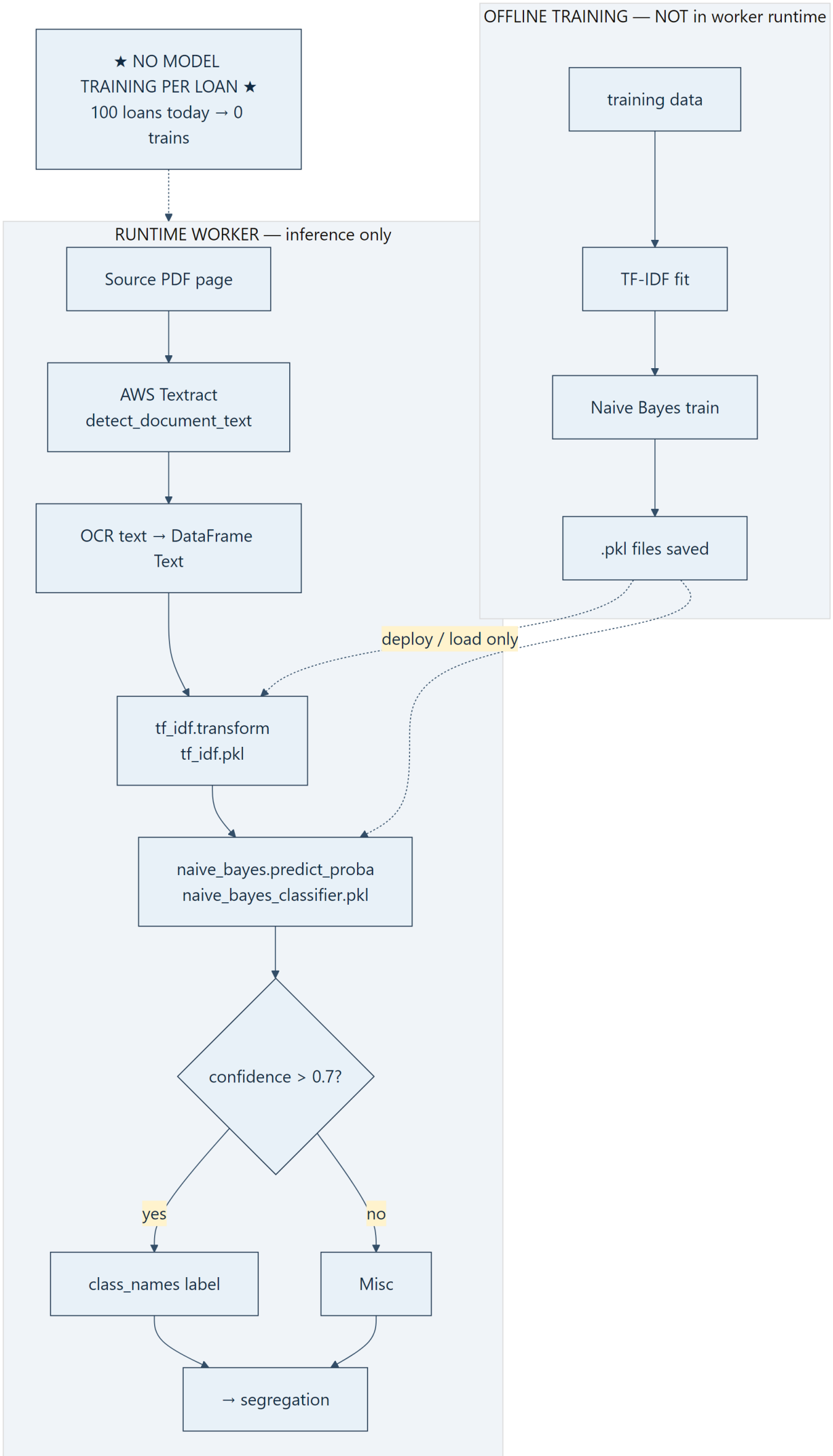
*Continued from previous page (same diagram — split only for readability).*




**KEY TAKEAWAY:** Failures mark Failed and still ack SQS — no automatic redrive proven in application code.

## Diagram 9. CLASSIFICATION — OFFLINE TRAINING vs RUNTIME

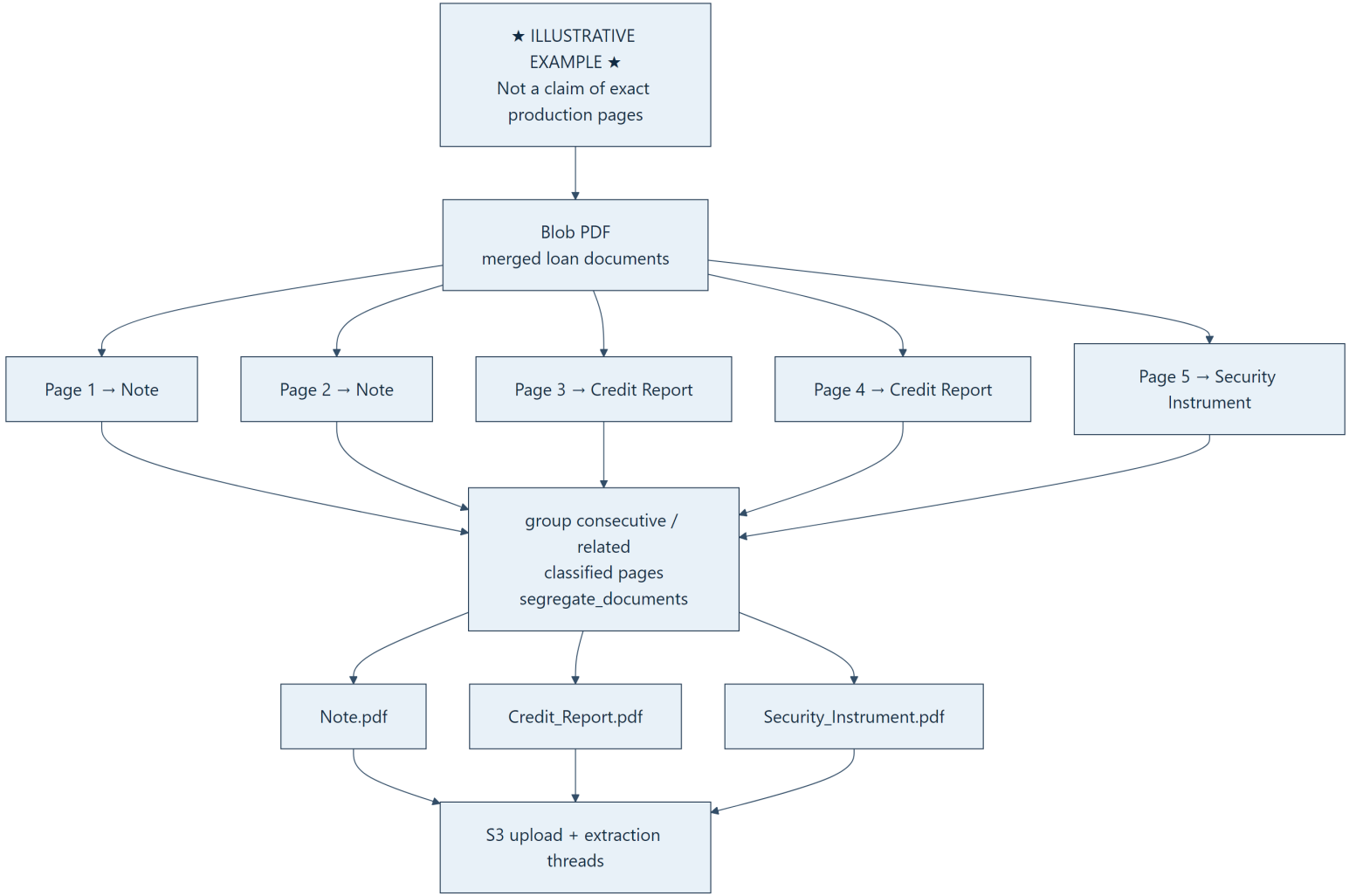
Runtime OCR for classification uses **AWS Textract** `detect_document_text`, then TF-IDF + Naive Bayes pickles via `predict_label` / `predict_proba`. Confidence ≤ 0.7 → `"Misc"`. Training is **not** in the worker — pickles are loaded artifacts. **Zero model training per loan.**




**KEY TAKEAWAY:** Worker loads and predicts. Training pipeline location is outside this runtime path (KT if needed).

## Diagram 10. SEGREGATION (ILLUSTRATIVE EXAMPLE)

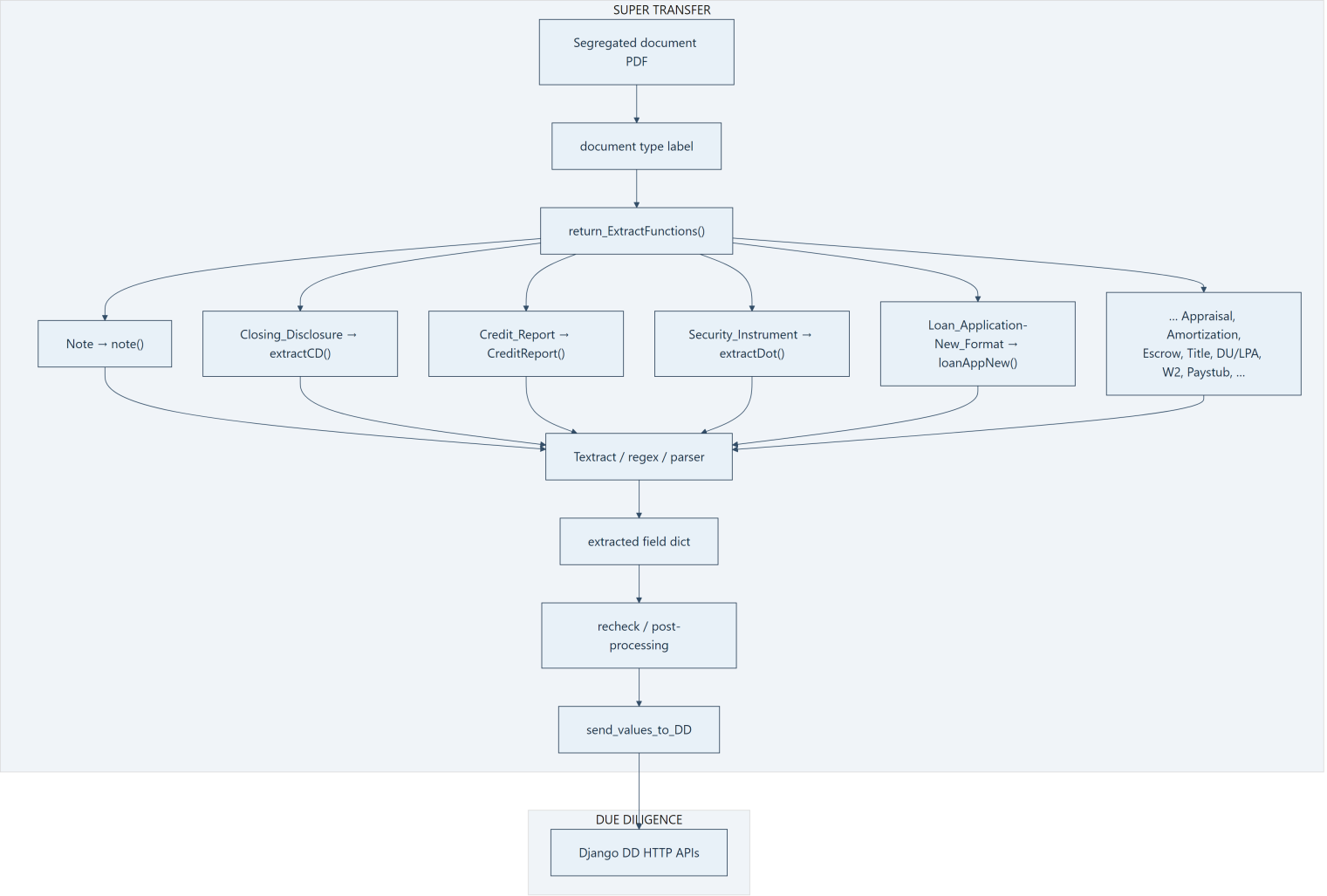
`segregate_documents` groups classified pages and splits the blob into per-type PDFs for extraction and S3 upload. Alternate path: classification JSON/txt sidecar → `zip_to_final_dict` skips full re-classification. Page labels below are **illustrative only**.




**KEY TAKEAWAY:** Segregation turns page labels into document-level PDFs that drive which extractors run.

## Diagram 11. EXTRACTION via `return_ExtractFunctions`

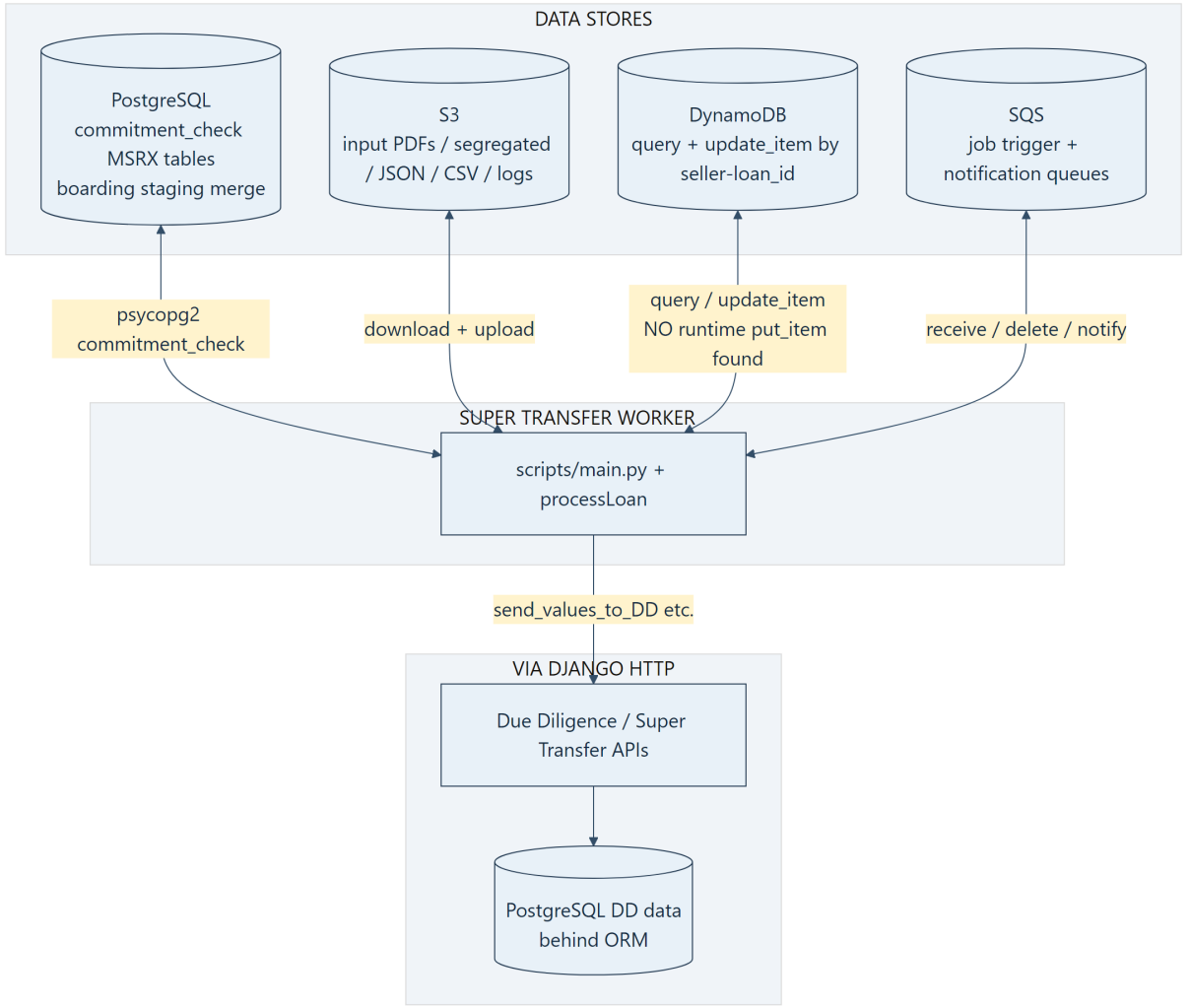
Each segregated label selects a confirmed extractor. Extractors use Textract / regex / parsers, then recheck/post-process, then `prepare_outputdict_and_values` → `send_values_to_DD`. Fields exist only when that document type was classified and its extractor ran.




**KEY TAKEAWAY:** Extraction is label-driven and partial — no universal field set for every loan.

## Diagram 12. DATA STORE INTERACTION (Worker-Centered)

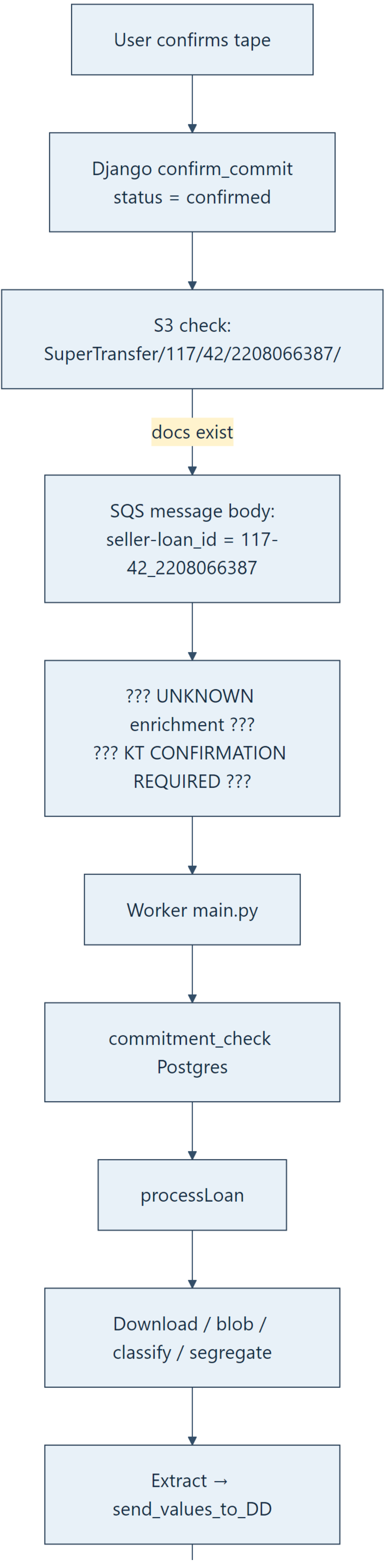
The worker talks to Postgres directly for `commitment_check` (and boarding staging merge SQL), to DynamoDB for loan state, to S3 for documents/outputs, to SQS for jobs/notifications, and to Django HTTP for Due Diligence values. Initial DynamoDB `put_item` is **not** in these repos' runtime paths → KT.




**KEY TAKEAWAY:** Worker is the hub for ST I/O; who **creates** the first DynamoDB item is still unknown.

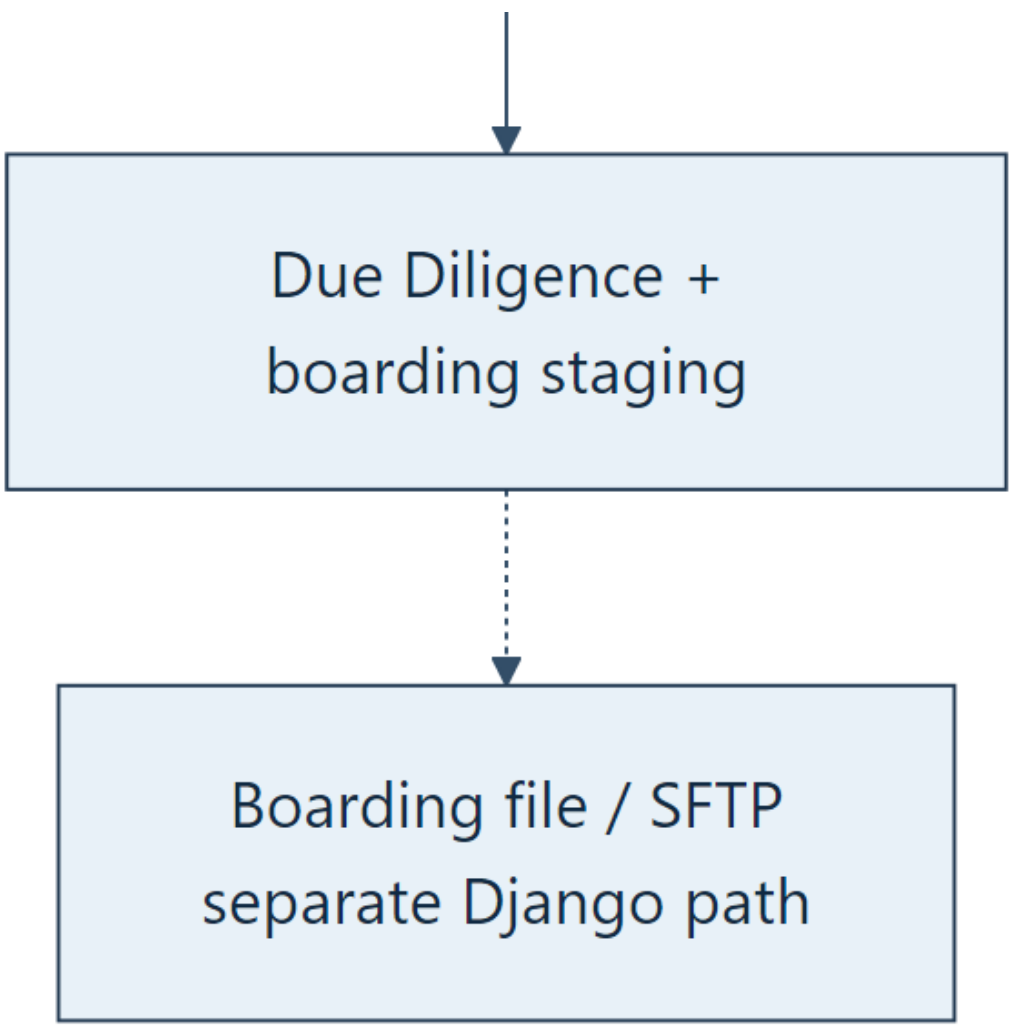
## Diagram 13. ONE-LOAN EXAMPLE — `117` / `42` / `2208066387` — Part 1/2

Easiest walkthrough for a new developer: same composite id from confirm through S3/SQS into the worker. Enrichment between SQS and `processLoan` remains unknown. Trader already saw commit-success while ST runs asynchronously.



## Diagram 13. ONE-LOAN EXAMPLE — `117` / `42` / `2208066387` — Part 2/2

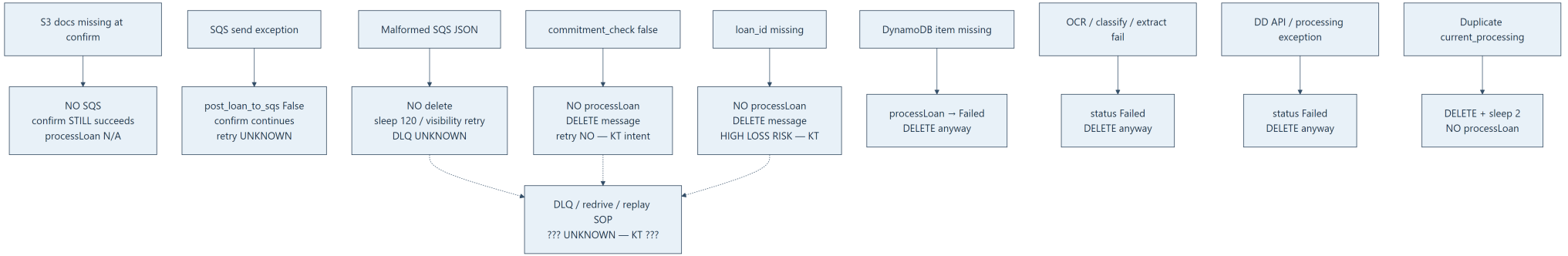
*Continued from previous page (same diagram — split only for readability).*




**KEY TAKEAWAY:** One composite id ties the loan across systems; async ST work continues after the trader’s success modal.

# Diagram 14. FAILURE FLOW (Confirmed Behavior Only)

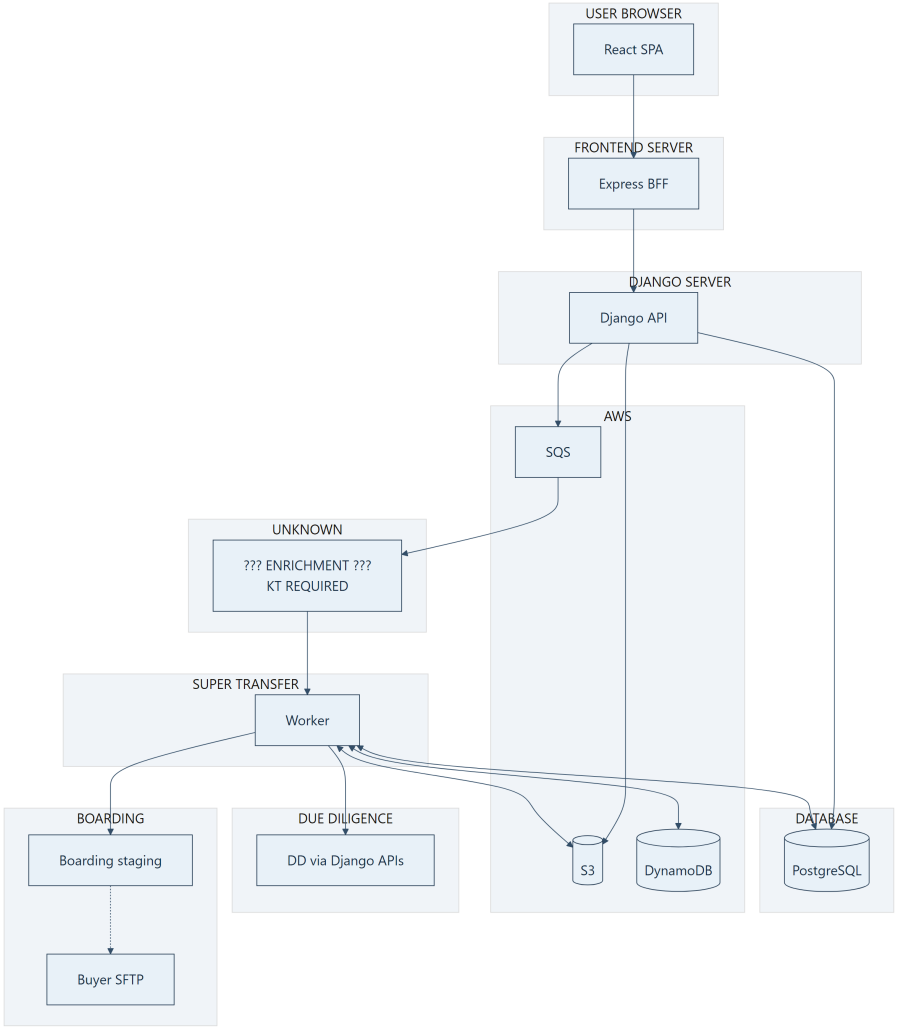
Only audit-confirmed outcomes. DLQ / redrive / ops recovery for delete-on-skip paths are **unknown**. Docs missing or SQS send fail → confirm can still succeed without enqueue.




**KEY TAKEAWAY:** Worst silent losses: skip enqueue at confirm, or delete-on-skip when `loan_id` / commit check fails — ask KT before operating incidents.

# Diagram 15. HIGH-LEVEL SYSTEM MAP (KT / Onboarding)

One-screen map of major boundaries only — no function overload. Use this first for architecture, then drill into diagrams 1–14.




**KEY TAKEAWAY:** Sync path ends at Django HTTP response; async path is SQS → unknown enrichment → worker → DD/boarding → (separate) SFTP.

## HOW TO READ THESE DIAGRAMS

1. **Start with Diagram 15** for architecture — major system boundaries only.
2. **Read Diagrams 2–3** for React → Express → Django confirm commit.
3. **Read Diagrams 4–5** for `seller-loan_id` and the SQS producer/consumer gap.
4. **Read Diagrams 6–8** for worker startup, SQS decisions, and `processLoan`.
5. **Read Diagrams 9–11** for classification, segregation, and extraction.
6. **Read Diagram 12** for databases / AWS interactions centered on the worker.
7. **Read Diagram 13** for a complete one-loan example (`117-42_2208066387`).
8. **Read Diagram 14** for confirmed failure behavior and KT gaps.

Treat every `??? UNKNOWN ???` box as a **question to ask**, not as a finished connector. For prose detail and evidence tables, use [`22_COMPLETE_END_TO_END_WORKER_FLOW.md`](./22_COMPLETE_END_TO_END_WORKER_FLOW.md).

**Reminder:** Treat every **??? UNKNOWN ???** box as a question to ask in KT — not as a finished connector. Do not assume Lambda unless confirmed.